Restoring Operational Histories with Synthetic Temporal Graph Supervision

Author details pending final review

Development manuscript: 15 October 2026

Internal development draft. The experiments below are development evidence. Independent confirmation, final ablations, bibliography reconciliation and reproducibility review are incomplete. This version is not ready for submission.

Abstract

Operational event records often contain enough information to describe what happened, yet unreliable object identifiers prevent users from retrieving a complete history of the affected order, application or asset. Missing participation, incorrect joins and fragmented identities interact: repairing one relation changes the evidence available for other decisions. We study restoration of typed event–object temporal graphs while retaining recorded activities, timestamps and legitimate exceptions. We develop executable workflow worlds that separate process behavior from recording faults and export learner inputs independently from repair references. The same execution supports repair and future-participation tasks. An auditable learned edit system retrieves candidate additions, removals and identity merges, updates the observed graph, and abstains when its development-selected criterion is unmet. Our development study compares temporal message passing, an MLP, boosted matching, sequential repair and source-data supervision. The simpler boosted edit learner is the strongest of the tested generated-data models; the temporal neural architecture does not establish an advantage. Generated-data training improves history recovery on an explored real-source log, but remains below a source-trained control and weakens under independently implemented recording faults. The final study will test the surviving claims on reserved process sources and report where executable training data helps, where it fails, and what information is required for dependable operational traceability.

1 Application and research question

An analyst investigating an application cancellation needs the records belonging to that application. A resource interaction assigned to the wrong application creates a false association. A missing application reference hides part of the history. If two systems use different identifiers for the same application, each identifier retrieves only a fragment. A high average link accuracy can therefore coexist with incomplete operational histories.

We ask whether supervision from executable workflow worlds can support useful restoration when clean operational histories are scarce. The application is offline reconstruction within an available observation window. It is not forecasting future links, changing the process to match a normative model, or inventing unrecorded activities. A canceled or retried operation is a fact to retain. The repair target is its association with the relevant entities.

Clean repair examples are difficult to obtain precisely because broken identifiers obscure which records belong together. Executable workflows offer a source of supervision, but a model can

learn the recording conventions of the simulator instead of transferable evidence. For example, a simulator's amount field may remain constant throughout an order, while an operational field records changing costs. A learned consistency rule can then remove valid records from a new process. This motivates our central question: does varying the observation of an unchanged execution improve the transfer of temporal graph repair?

We contribute an executable supervision protocol that separates workflow execution, observed fields and recording faults; a paired study of observation variation for coupled participation and identity edits; and an application evaluation based on retrieved histories, preserved records and false associations. The repair system uses established classifiers and graph operators. The contribution is the training-data construction and its measured application behavior, rather than a new neural architecture. Its bounded generator component also exports a future-participation task from the same execution, providing a reusable interface without making the general dataset factory part of this paper.

2 Related work and comparison boundaries

HERE combines Transformer prediction and domain rules for missing case identifiers [1]. Its native output is a single case identifier. SANAGRAPH reconstructs missing attributes within supplied traces through heterogeneous graph learning [2]. An unchanged trace-based model cannot be supplied with correct trace membership in a benchmark where that membership is itself damaged. DDTR uses diffusion for activity-trace recovery [3]; its generative formulation likewise does not make it an unchanged entity-identity repair baseline.

Swevels et al. use contextual Event Knowledge Graphs and physical inference rules to recover missing identifiers [8]. Their implementation uses domain semantics such as loading, unloading and physical location. Object-centric imperfection patterns characterize identity and association problems [9]. We study their coupled effects on history retrieval when such physical rules and clean target histories are unavailable. This differs from originating the identifier-repair problem itself.

HoloClean combines heterogeneous signals for probabilistic data repair [4]. Our implementation likewise uses multiple sources of observed evidence, but learns its edit scores from generated workflow references and evaluates entity-history recovery. Comparisons include observation-only boosted matching, target-local pseudo supervision, fixed-sequence repair and neural edit models. The heterogeneous GraphSAGE control adapts the standard aggregation operator [5] to event/entity inputs and typed edits; it is not an exact SANAGRAPH reproduction.

Object-centric simulation also predates this work [6]. A recent independent collection provides six synthetic relational systems with reference object-centric event logs [7]. We use this collection to challenge simulator dependence rather than claiming to introduce multitype operational simulation. The complete general dataset factory remains outside the scope of this paper.

Synthetic training and observation randomization are established approaches to transfer [12]. PluRel and RDB-PFN generate relational databases for model training and transfer to relational prediction tasks [13, 14]. TIGGER learns to generate temporal interaction graphs [15]. We apply synthetic supervision to a different target: edit decisions when the available event–object associations themselves are unreliable. The evaluation asks whether generated supervision restores useful histories and how often repair introduces false associations. It does not measure fidelity to a learned real-graph distribution.

FLAWD provides a language and tool for controlled event-log fault injection [10], and algebraic repair expressions have dedicated language support [11]. Fault scenarios and auditable edits serve as supporting components here. The paired observation experiment holds execution and fault

references fixed so that their contribution can be distinguished from the choice of repair scorer.

3 Task and information boundary

Let an event be $e = (u, t, a, P, x)$, where u is an opaque event handle, t is occurrence time, a is activity, P is a set of typed object handles and x contains legitimate observed attributes. The entity catalog supplies handle types. The observed temporal graph contains event–object incidence and temporal adjacency derived from observed histories. Equal-time records are simultaneous; source row order is not treated as a causal order.

The repair system receives the damaged graph and a declared target-type schema. It does not receive the true identity map, original case containers, missing-edge locations, fault seed or clean derived adjacency. Allowed changes are `ADD_PARTICIPANT`, `REMOVE_PARTICIPANT` and `MERGE_ALIASES`. Activities, timestamps and descriptive attributes are protected. A merge identifies same-type handles and deduplicates their participation. The system may stop with unresolved uncertainty.

The entity catalog is assumed available. This is a material assumption: reconstructing entirely unknown entities is a different open-catalog problem. Likewise, offline context may include later records in the declared window. It must not be compared directly with a strictly online predictor receiving only past events.

The controlled faults affect declared target types. Other observed relations, such as resources or vendors, supply context and are left unchanged. This is a scoped recording-error model: the experiments do not establish robustness when every contextual relation or timestamp may also be unreliable. No type is supplied with a trusted canonical identity map.

Reference graphs are evaluator-only. Simulated executions provide exact symbolic references. Public-source logs provide controlled-corruption references, not an independent certification that every native record is correct. Attributes copied from source trace metadata into event records form an explicitly denormalized observation condition; they must be reported separately from native-event and structural-only conditions.

4 Executable training worlds

4.1 Execution, observation and task construction

A world executes a finite workflow with parents, work items, resources and optional batches. Mechanisms include branching and joining, resource queues, handoffs, batching, retries, cancellation and return. The expanded configuration adds finite multistage work routes, repeated work, start/completion records and waiting-time variation. Resource-capacity constraints are enforced when queueing is enabled. These are symbolic mechanisms, not a calibrated organizational digital twin.

Observation projection selects visible entity types and descriptive fields before faults are applied. Clean and damaged variants share execution ancestry. The repair task exports a public learner graph and separate reference participation, identity and fault records. A second task exports available prefix observations and future-participation targets from the same execution, excluding future-born entities from the candidate catalog. This reuse is a bounded interface check, not evidence for a universal dataset platform.

Each execution records its normalized recipe, random states, source fingerprint and pending work. A JSON checkpoint resumes to the same execution as an uninterrupted run. Export manifests

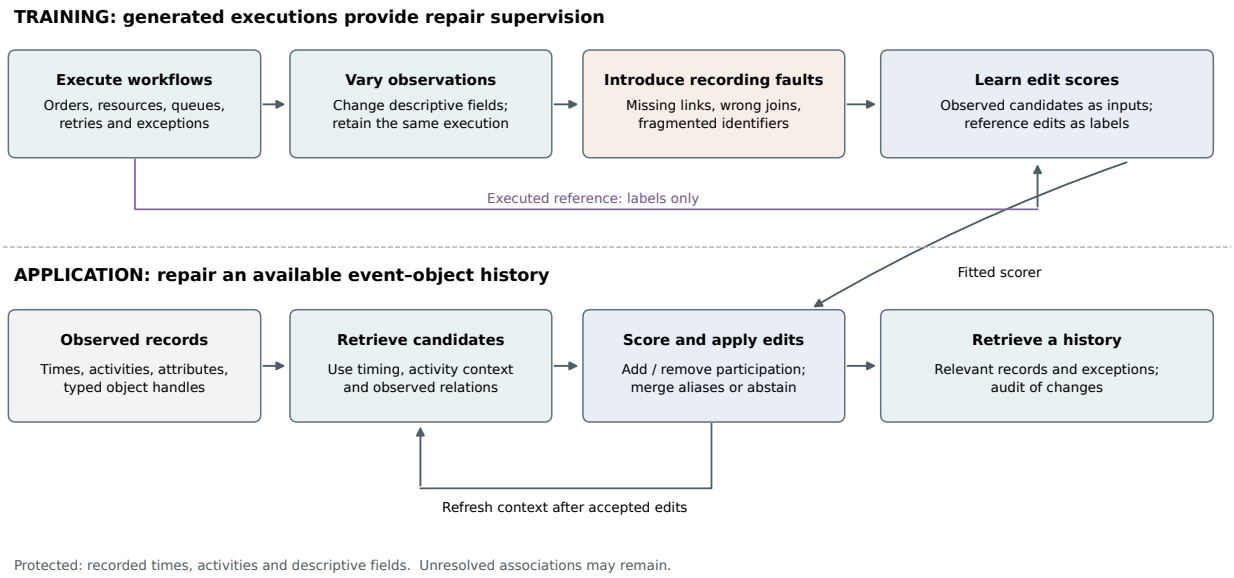


Figure 1: The implemented training and application boundary. Executed references supervise training labels. At application time, candidate retrieval and edit scoring use the observed temporal graph. The system retains recorded event content and can leave uncertain associations unresolved.

hash learner, reference and metadata files; observations derived from one execution remain in the same split. Changing source code invalidates incompatible checkpoints.

4.2 Paired observation generation

The principal comparison changes descriptive fields while keeping executed events, visible entity types, timestamps and corruption references fixed. Each world receives one of four seeded observation profiles: original fields; activity-dependent categories; numerical measurements; or a mixture of activity categories, measurements, incidental categories and time bands. Fields can be sparse. Numerical measurements combine an existing amount, when available, with activity-dependent scales and noise; no field is derived from a target identifier. Every clean/damaged pair receives the same aligned event descriptions through builder-only execution ancestry. This is an application of observation randomization, not a new general randomization principle.

The comparison holds world count, execution recipes, fault realizations, sampled query events, learner capacity and generated calibration collection fixed within each trial. Observation changes can alter candidate retrieval and therefore the number of unique candidate pairs. It is consequently not an exactly equal number of binary training examples. Independent repetitions regenerate the executed worlds and their observation fields.

4.3 Variation and held-out conditions

Training and generated validation use disjoint mechanism compositions. Vocabulary profiles vary independently of flat versus multi-object projection. The profiles share one simulator engine and therefore do not constitute independent domains. History length, resource count, waiting scales, attribute collisions and error severity vary independently within the declared schedule.

The original recording faults combine contiguous or independent missing participation, same-type incorrect joins and time-cut identity fragmentation. An independently implemented development stressor uses activity-routed namespaces, a recording outage over a time interval and persistent stale join mappings. Its rates are declared experimental conditions, not estimates of real incident frequencies. Both fault families preserve recorded activity and time.

The reserved mixed condition fragments eligible histories of at least four records with probability .25, removes approximately .10 of each resulting handle's participation in a contiguous block, and replaces remaining target participations with probability .10 by a different same-type entity. Stochastic rounding includes short histories in the missing-link process. The pipeline condition routes activities between two namespaces, selects eligible entities for fragmentation with probability .25, removes target links over a time interval covering approximately .10 of the ordered records, and gives selected entities a persistent incorrect join over half their surviving history. The latter selection probability is .25. Equal timestamps stay together at the outage boundary. Actual damaged-event counts are reported because these mechanisms do not produce identical corruption severity.

5 Auditable learned graph edits

Candidate retrieval combines temporal proximity, activity context, observed co-participation and descriptive agreement. Candidate recall is reported separately from classifier performance. Handles are anonymous identifiers, never numerical predictors. Cold objects with no evidence remain difficult or unidentifiable; reference knowledge does not make them recoverable from the learner input.

For participation candidates, the learner uses degree, normalized context overlap, temporal bracketing, activity transitions and attribute agreement. The queried event is excluded from supporting history when assessing its current association. Identity candidates use symmetric history and context features, including overlap, activity profiles, observed attributes and temporal boundary compatibility. Overlapping lifetimes are permitted in candidate retrieval.

Training references label candidate edits and create valid partial-repair states. They are never inputs to the candidate constructor or inference scorer. The boosted control uses separate conditional action classifiers. It is a discriminative edit learner trained with generated supervision, not a deep generative density model. Neural controls use the same candidate action features and graph-derived node inputs, with temporal message passing, relation-specific mean GraphSAGE or no graph messages.

At inference, the system scores candidate edits, selects a commuting batch, applies it and recomputes evidence. A batch cannot merge an entity while also modifying its participation. The implementation records scores and reversible changes. Development thresholds and the pass limit determine abstention; the thresholds do not provide an out-of-domain precision guarantee. One-pass and fixed-sequence variants isolate the benefit of revisiting the graph. Repeated source-trained matching is also a required comparator.

The generated reference labels a merge positive when its handles refer to the same entity, a removal positive when the recorded participant is incorrect, and an addition positive when it supplies a missing reference entity. These are local edit labels, not an optimization guarantee for final history F1. Training also includes partial teacher-repaired states to expose the learner to updated context. At inference, an accepted participation edit cannot subsequently be reversed and an identity merge cannot be split. Although the audit can undo a batch, the learned decoder has no search or backtracking policy; an early mistake can therefore persist.

6 Evaluation design

6.1 Sources and splits

BPI 2012 [16] and the public simulated logistics log are development sources. BPI 2012 cases are assigned to fit, tune and reserved check partitions before anonymization. Shared resource types are allowed, but held-out events do not enter another partition's observation context. BPI 2017 [17] and BPI 2019 [18] cohorts were reserved until the models, calibration caches, adapters, faults and comparisons were frozen. Their confirmation experiments are now running.

BPI 2017 contains applications and explicitly recorded offers from the same institution as BPI 2012, after a supporting-system change. It does not establish transfer to an unseen organization. We select 1,024 applications by seeded hash rank after excluding the first 500 applications inspected for feasibility and their offers. The resulting cohort contains 38,822 events and 2,422 target entities (1,024 applications and 1,398 offers). Only explicit event-level offer references become offer participation; absence of an offer reference is not a certified absence of a real-world offer relationship.

For BPI 2019 we use the released OCEL conversion [19, 20], selecting 1,024 purchase orders after excluding orders and items occurring in the first 2,000 feasibility-audited events. The cohort contains 20,396 events and 4,325 target entities (1,024 orders and 3,301 items). Resource and vendor relations remain observed context. Here "native" refers to the released OCEL representation, not a claim that every relation was an explicit field in the original XES event. Copied trace attributes, identifier fields and object-to-object relations are removed from learner input. The retained descriptive event field is cumulative net worth. The reserved BPI 2012 check partition contains 42,366 events and 2,034 application histories; its copied requested amount is explicitly labeled as denormalized.

The expanded executable collection contains 512 training worlds and 96 validation worlds. The independent relational/OCEL collection uses BatteryVault and ClaimStream as development domains, selected by alphabetical order before record inspection; its remaining four scenarios are reserved for confirmation. These are external simulations, not additional real-world datasets.

Final experiments must distinguish synthetic-only learning, source-only learning, mixed learning and any target-data adaptation. For limited source annotation, a verified history supplies its event memberships and alias handles. All available annotated positive pairs may train the matcher even if candidate retrieval missed them; inference retrieval remains label-free. Development label budgets must not be described as total annotation costs when additional labeled validation data select thresholds.

6.2 Application outcomes

For an observed handle u, its predicted cluster retrieves an event set $\widehat{H}(u)$. Let H_q be the reference history of entity q and A_q its observed handles. The primary metric is

$$F_{\text{history}} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{|A_q|} \sum_{u \in A_q} \frac{2|\widehat{H}(u) \cap H_q|}{|\widehat{H}(u)| + |H_q|}.$$

Only reference entities with a recorded history enter Q. This first averages over an entity's aliases and then equally over entities, preventing heavily fragmented entities from dominating. Exact-history retrieval replaces the inner F1 by $1[\widehat{H}(u) = H_q]$ with the same weighting; it is not a requirement that every alias of an entity succeed simultaneously. We also report history precision/recall and per-type results. Identity-pair precision and recall expose false merges and unresolved fragments.

Operational queries measure whether a handle retrieves evidence of a cancellation-related event and whether the first and last recorded timestamps are correctly recovered. Matching a boundary timestamp does not imply that every simultaneous boundary event is restored. These queries concern recorded evidence, not a causal judgment that an entire business transaction was canceled. Correct-event preservation, cold-history coverage, genuine exception associations, candidate coverage and runtime accompany aggregate repair scores.

7 Generated confirmation

The frozen compositional test contains 96 worlds from two mechanism combinations absent from both training and validation. The mechanisms and simulator engine are shared. All six learners evaluate the same worlds and recording faults. Across three independently generated training collections, observation variation raises mean history F1 from .7588 to .8000, compared with .7340 before repair (Figure 2). The three paired improvements are .0464, .0412 and .0359. The observed trial range is .7554–.7650 for original fields and .7913–.8113 for varied fields. These are descriptive ranges across training collections, not event-level confidence intervals.

Exactly restored histories increase from .0090 before repair to .0357 with original fields and .1098 with varied fields. Thus most histories remain incomplete even when average F1 improves. Varied-field training improves 89, 90 and 90 of the 96 damaged worlds and worsens 7, 5 and 5, respectively. On the paired clean worlds its mean history F1 is .999867, with 16, 14 and 21 edits across the three trials; 7, 3 and 3 clean worlds worsen. The result supports useful compositional recovery while retaining evidence of unnecessary edits.

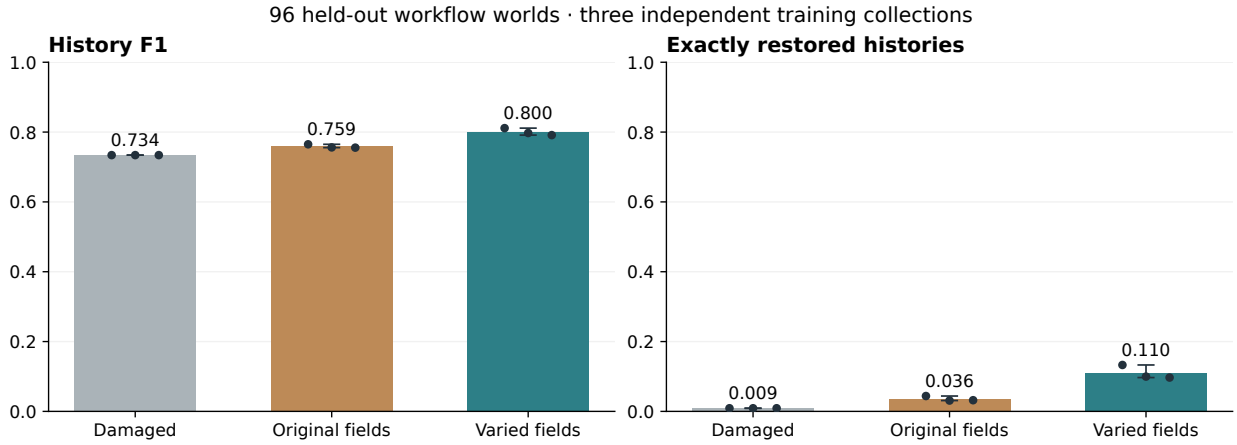


Figure 2: Frozen generated confirmation. Each dot is the equal-world average from one independent training collection; bars show their mean and whiskers their observed range. All trials use the same 96 held-out worlds. Complete-history recovery remains substantially below average history F1.

8 Confirmation on operational sources

The loan cohort provides a complete three-trial comparison. With mixed recording faults, mean history F1 increases from .7872 before repair to .7914 under original generated observations and .8050 under varied observations. The paired varied-observation advantage is positive in all three trials. Exact-history retrieval increases from .1696 to .1865. The varied learner introduces no false

identity pairs in these trials, but recovers only .0105 of the true identity pairs on average. Most of its benefit therefore comes from participation repair, rather than complete recovery of fragmented identifiers.

The BPI 2012 source-supervised edit model transfers poorly to the loan cohort, reaching mean F1 .5472 and damaging thousands of originally correct event associations. Target-local extra-corruption training mostly abstains. These controls distinguish transfer from having useful supervision in the same process: the strong source-development results in Appendix C do not imply reliable cross-process transfer.

The independent pipeline fault family exposes a different outcome. Loan history F1 decreases in every trial, from mean .7785 to .7771, with zero correctly recovered identity pairs. This failure remains part of the confirmation result. On uncorrupted loan references, F1 is .999887 on average, but 31, 10 and 34 event assignments become incorrect. High clean-set F1 must therefore be interpreted together with the counts of damaged records.

9 Application interpretation and limitations

The intended output is a reconstructed history and an inspectable record of suggested changes. It can help an analyst recover evidence spread across identifiers, but the current operating point does not establish safe unattended repair. The loan example in Appendix D restores an offer-creation record while retaining cancellation evidence, yet a short offer history loses a correct record. The examples illustrate outcomes already counted in the aggregate metrics.

Initial candidate coverage helps locate a failure before the learned scorer is applied. Across the three loan mixed-fault trials, the correct application is retrievable for .955–.982 of sampled missing application links. Under the pipeline faults, this falls to .496–.526. Initial offer-link coverage is .750–.862 for mixed faults and .420–.591 for pipeline faults. Identity-pair coverage is higher under the pipeline condition, yet the decoder recovers no true pairs there. Candidate availability is thus necessary but insufficient: identity evidence, learned scores and abstention also matter. These initial-state diagnostics are not fixed upper bounds, because later edits can change retrieval.

The generated worlds share one symbolic engine. The held-out test assesses new compositions of known mechanisms, not organizational realism. BPI 2017 comes from the same institution as BPI 2012, whereas BPI 2019 supplies another application family through a released OCEL conversion. Controlled corruption references are reproducible but do not certify native records or substitute for independently annotated natural errors. Protected event content, a typed catalog and reliable non-target context are material task assumptions.

Three primary trials jointly vary generated training worlds and test corruption seeds. Paired comparisons hold both fixed within a trial, but their separate variance components are not identified. Single-trial neural, structural and decoding controls support bounded mechanism comparisons, not broad architecture rankings. Neural training samples actions while boosted training uses all cached candidates. Runtime was measured under shared concurrent load and is not presented as a controlled speed benchmark. The reference labels identify locally correct edits; neither their classifier loss nor the irreversible decoder guarantees the best final history.

A Implementation details

The principal learner fits one histogram gradient-boosting classifier per edit operation, with 200 boosting iterations, at most 15 leaves, learning rate .08 and L_2 regularization 1. Early stopping is disabled and the fitting seed is zero. Positive and negative examples each receive half the aggregate

training weight. This fixed learner isolates the effect of generated observations; it is not proposed as a new learning architecture.

Participation scoring uses 18 history-matching features plus observed type/activity prevalence and same-type event degree. Identity scoring uses 25 symmetric history-matching features. These are placed in a shared 28-dimensional action vector and clipped to $[-10, 10]$. Retrieval and feature construction exclude the queried event from its supporting history. Training samples up to 48 events per state, 16 participation candidates per event/type and 32 identity candidates per handle. Candidate generation never receives the reference labels. The 512-world training collection supplies 896 states: 512 damaged states, 128 clean states and 256 partial-repair states. Generated validation supplies 120 states from 96 worlds.

For each operation, the threshold maximizes empirical recall among generated-validation thresholds with empirical precision at least .95 and at least five true positives. If none meets this criterion, that operation abstains. Scores from class-weighted training are not asserted to be calibrated probabilities. Threshold selection is a finite validation rule, with no statistical precision guarantee on a new process. The real-source and independent-simulator protocol permits four passes with participation/pair budgets 128/64; the generated compositional test uses eight passes and budgets 16/32. Thus the two test regimes are reported separately.

Neural controls use width 64, two message-passing layers where applicable, 16-dimensional node features, type/activity embeddings and three edit heads. GraphSAGE uses a root projection and relation-specific mean-neighbor projections. Its adjacency includes event–object incidence and temporal neighbor relations; removing numerical edge-time inputs does not make it a time-free model. The temporal control additionally uses signed relative-time edge features and learned message gates. The MLP omits message passing but retains shared temporal node/action inputs. All three controls use the same precomputed candidate universe and generated collections. Neural training samples at most 2,048 actions per state, including at most half positives; boosted training uses all cached candidates. This optimization-budget difference limits architectural conclusions.

The revised neural controls train for 16 epochs with AdamW (learning rate .001, weight decay .0001), cosine decay to .0001, gradient clipping at 5 and dropout .1. The mean generated-validation average precision across operations selects the checkpoint; epoch 10 was selected for all three revised controls. Positive loss weights are computed from training frequencies and capped at 30. No reserved-target labels select checkpoints or thresholds.

B Candidate coverage and reproducibility

The local review artifact contains the generated test worlds, model checkpoints, generated calibration scores, source snapshots, trial reports and saved edit programs. A relocated copy passes 51 observation-boundary, decoder, adapter and generator checks. Replaying the fixed first generated graph independently for each of the six primary models reproduces its full application outcomes and exact edit program. Rebuilding the 14 BPI 2012 confirmation tasks in the relocated tree reproduces all 28 observation/reference file hashes. These are same-environment reproducibility checks, not independent retraining evidence.

The original protocol remains unchanged, identified by SHA-256 beginning `a3e12b092cc7670f8`. Protocol relocation changes only declared source paths and the output task directory, verifies source bytes, and records both protocol hashes. Interrupted processes resume completed rounds; runs interrupted before their first checkpoint archive the empty partial record and replay the same configuration. Generated evaluations retain completed graph outcomes and continue in the original order. No observed result is removed to improve an aggregate.

Source	Candidate target	Mixed	Pipeline
BPI 2017	Missing Application	0.968 [0.955, 0.982]	0.512 [0.496, 0.526]
BPI 2017	Missing Offer	0.809 [0.750, 0.862]	0.481 [0.420, 0.591]
BPI 2017	Identity pair: all	0.681 [0.671, 0.691]	0.898 [0.877, 0.916]
BPI 2017	Identity pair: both observed	0.717 [0.700, 0.736]	0.943 [0.913, 0.972]
BPI 2019	Missing PO	0.999 [0.998, 1.000]	0.955 [0.925, 0.978]
BPI 2019	Missing POItem	0.982 [0.976, 0.987]	0.926 [0.880, 0.953]
BPI 2019	Identity pair: all	0.906 [0.902, 0.913]	0.906 [0.882, 0.926]
BPI 2019	Identity pair: both observed	0.952 [0.947, 0.955]	0.949 [0.937, 0.956]
BPI 2012 check	Missing Application	0.979 [0.974, 0.985]	0.677 [0.667, 0.696]
BPI 2012 check	Identity pair: all	0.738 [0.719, 0.769]	0.892 [0.875, 0.912]
BPI 2012 check	Identity pair: both observed	0.744 [0.725, 0.773]	0.931 [0.911, 0.953]

Table 1: Initial candidate recall, mean and observed range over three fault trials. Missing participation is audited on 2,048 uniformly sampled events per task; identity coverage enumerates candidate pairs. Both-observed identity recall excludes pairs with a handle lacking an observed history. Coverage measures retrievability, not identifiability or final recovery.

C Development evidence motivating observation variation

These results influenced method selection and are exploratory. On the BPI 2012 tune partition, the original generated learner raises history F1 from .7445 to .8380. A participant matcher trained on source histories reaches .8922, adding source-trained identity matching reaches .9107, and refreshing both reaches .9164. Those matching controls also exploit the source task's one-application-per-event contract. They are useful within-process reference points, not unchanged multi-object baselines.

At 16, 64 and 256 verified source training histories, source-only edit learning reaches .8359, .8321 and .8753; adding generated examples reaches .8912, .8905 and .8896. All use additional source-development calibration labels. These single-selection results suggest a possible augmentation benefit, but do not establish the total annotation cost or a monotone label-efficiency curve.

On the independent BatteryVault and ClaimStream simulations, the original generated learner fails with native fields (.7371 and .7519), below the damaged inputs (.7682 and .7617). Hiding descriptive attributes as a diagnostic reaches .8543 and .8844. Changing generated calibration alone gives .7964 and .7978. Paired varied-observation training reaches .8804 and .9197 in the initial trial (Figure 3). Two independent training collections give .8861/.8452 for BatteryVault and .9195/.8603 for ClaimStream. The respective three-trial means are .8706 and .8999; all use the same development fault realization.

The revised learner's initial trial makes zero edits on the uncorrupted counterparts of both simulations. Under mixed faults, exception-evidence F1 reaches .9247 and .9648. Under pipeline faults, history F1 reaches only .8075 and .7985, including 7 and 26 false identity pairs. Varied-observation training also lowers BPI 2012 tune F1 from .8380 to .8074. The automatic stable-field alternative reaches .8393 on BPI 2012 and .8588/.8920 on the simulations. These trade-offs motivated retaining the stable-field control and per-source reporting in confirmation.

The original structural exception audit was corrected after discovering that its positive-event set had been derived from hidden learner fields. The corrected audit uses evaluator-only exception flags from the pinned native observations; history results are unchanged. Reports retain both the original record and the corrected query evidence.

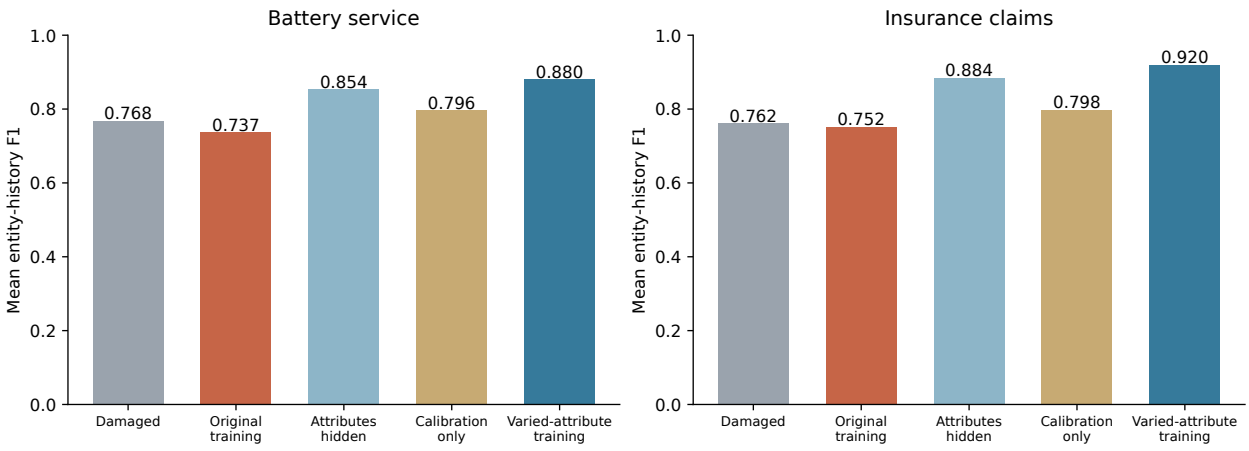


Figure 3: Development diagnostic on the same independent-simulator records and faults. Hiding attributes is a manual diagnostic. Calibration-only retains the original fitted model, while varied-observation training changes generated descriptions with executed workflows and fault labels held fixed. The figure shows the initial trial, not the later confirmation cohorts.

D Illustrative application audit

Figure 4 shows two post hoc examples from BatteryVault development. The success case is an exception-bearing history whose improvement is nearest the median positive improvement among such histories. The failure case has the largest history-F1 loss. Ties use a hash of the opaque identifier. These illustrations explain behavior and are not separate evidence of generalization.

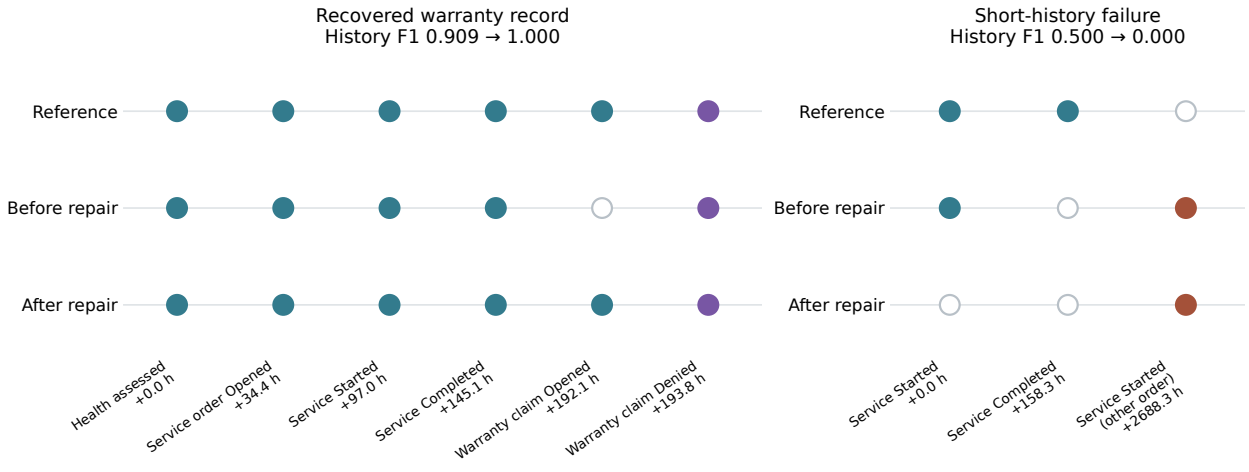


Figure 4: A recovered warranty record and a failed short service history. Filled circles are retrieved records; empty circles are absent. Purple denotes a genuine exception, and rust a record belonging to another order. Times are relative to the first displayed record, with chronological but evenly spaced columns. Entity scores average over aliases; the diagram displays one hash-selected handle.

The same case-selection rule applied to reserved BPI 2017 trial zero identifies the application in Figure 5. Its full reference history contains 26 records. Repair removes an unrelated call record and restores an offer-creation record, increasing history F1 from .8980 to .9388 while retaining the

recorded application and offer cancellations. Three reference records remain missing. Across that trial, 1,036 entity histories improve, 79 worsen and 1,307 remain unchanged. The paired failure has only two reference offer records: its correct creation association is removed, its cancellation remains missing and an unrelated creation record remains attached. These examples make both the operational benefit and the risk of automatic short-history repair concrete.



Figure 5: Post hoc illustrations from reserved loan trial zero. The application panel shows changed associations and cancellation records; its score uses the full history, including records omitted from the diagram. The short offer panel shows the complete union of reference and retrieved records. Purple marks retained or missing cancellation evidence; rust marks an unrelated entity's record. Columns are chronologically ordered and evenly spaced, with relative hours shown.

References

[1] F. Zetzsche et al. *Case ID Revealed HERE: Hybrid Elusive Case Repair Method for Transformer-Driven Business Process Event Log Enhancement*. Business & Information Systems Engineering 67, 311–337, 2025. https://doi.org/10.1007/s12599-025-00935-5.

[2] S. Dissegna, C. Di Francescomarino and M. Ronzani. *Graph-Based Event Log Repair*. CAiSE 2026, 423–440. https://doi.org/10.1007/978-3-032-28117-3_24. Accessible author preprint: https://arxiv.org/abs/2508.05145.

[3] M. Matyash, A. Gal and A. Senderovich. *DDTR: Diffusion Denoising Trace Recovery*. 2025. https://arxiv.org/abs/2510.22553.

[4] T. Rekatsinas, X. Chu, I. F. Ilyas and C. Ré. *HoloClean: Holistic Data Repairs with Probabilistic Inference*. 2017. https://arxiv.org/abs/1702.00820.

[5] W. L. Hamilton, R. Ying and J. Leskovec. *Inductive Representation Learning on Large Graphs*. Advances in Neural Information Processing Systems 30, 2017. https://arxiv.org/abs/1706.02216.

[6] B. Knopp, M. Pourbafrani and W. M. P. van der Aalst. *Discovering Object-Centric Process Simulation Models*. ICPM 2023. https://ieeexplore.ieee.org/document/10271944/.

[7] A. Berti. *Synthetic relational databases and reference OCELs*. Zenodo, version 1, 2026. https://doi.org/10.5281/zenodo.22079246.

[8] A. Swevels, R. Dijkman and D. Fahland. *Inferring Missing Entity Identifiers from Context Using Event Knowledge Graphs*. BPM 2023, 180–197. https://doi.org/10.1007/978-3-031-41620-0_11.

[9] S. Sadeghianasl et al. *Object-centric event-data imperfection patterns*. Information Systems 141, 102766, 2026. https://doi.org/10.1016/j.is.2026.102766.

[10] M. Comuzzi, J. Ko and F. M. Maggi. *A Language to Model and Simulate Data Quality Issues in Process Mining*. ACM J. Data Inf. Qual. 17(2), Article 6, 2025. https://doi.org/10.1145/3743144.

[11] S. Sadeghianasl, R. Dijkman, R. Andrews and A. Barros. *A Language for Efficient Event Log Repair*. EDOC 2026. Conference record: https://conf.researchr.org/details/edoc-2026/edoc-2026-papers/8/A-Language-for-Efficient-Event-Log-Repair.

[12] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba and P. Abbeel. *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*. 2017. https://arxiv.org/abs/1703.06907.

[13] V. Kothapalli, R. Ranjan, V. Hudovernik, V. P. Dwivedi, J. Hoffart, C. Guestrin and J. Leskovec. *PluRel: Synthetic Data unlocks Scaling Laws for Relational Foundation Models*. 2026. https://arxiv.org/abs/2602.04029.

[14] Y. Wang, J. You, C. Shi and M. Zhang. *Relational In-Context Learning via Synthetic Pretraining with Structural Prior*. 2026. https://arxiv.org/abs/2603.03805.

[15] S. Gupta, S. Manchanda, S. Bedathur and S. Ranu. *TIGGER: Scalable Generative Modelling for Temporal Interaction Graphs*. AAAI 36(6), 6819–6828, 2022. https://doi.org/10.1609/aaai.v36i6.20638.

[16] B. F. van Dongen. *BPI Challenge 2012*. Dataset, 2012. https://doi.org/10.4121/uuid:3926db30-f712-4394-aebc-75976070e91f.

[17] B. F. van Dongen. *BPI Challenge 2017*. Dataset, 2017. https://doi.org/10.4121/uuid:5f3067df-f10b-45da-b98b-86ae4c7a310b.

[18] B. F. van Dongen. *BPI Challenge 2019*. Dataset, 2019. https://doi.org/10.4121/uuid:d06aff4b-79f0-45e6-8ec8-e19730c248f1.

[19] S. Khayatbashi, O. Hartig and A. Jalali. *Transforming Event Knowledge Graph to Object-Centric Event Logs: A Comparative Study for Multi-dimensional Process Analysis*. ER 2023, 220–238. https://doi.org/10.1007/978-3-031-47262-6_12.

[20] S. Khayatbashi and A. Jalali. *BPI Challenge datasets—OCEL 2.0*. Zenodo, version 1, 2026. https://doi.org/10.5281/zenodo.22663565.